

## ML Models & Input/Output

FinPulse will use a **multi-model architecture** with five ML components. For each model we define its inputs, outputs, and suggested algorithms:

- **1. Financial Health Score (Regression):**

- *Inputs:* Borrower profile + aggregated banking metrics – e.g. monthly income, total expenses, savings rate, cash-flow volatility, EMI/Income ratio, average balance, credit history length.
- *Outputs:* A numeric *Health Score* (0–100) and a categorical risk label (Low/Medium/High).
- *Approach:* A supervised regression model (e.g. **XGBoost** or **Random Forest**) is ideal for handling non-linear patterns in tabular data. Gradient-boosted trees are “excellent” for credit features <sup>1</sup>. (Logistic regression can serve as a simple baseline).

- **2. Loan Default Risk (Binary Classification):**

- *Inputs:* The Health Score (above) plus other risk indicators – e.g. missed EMIs count, EMI/Income ratio, income variance, expense spikes, number of active loans.
- *Outputs:* A default probability (0–1) and a label (Safe / Watch / High Risk).
- *Approach:* A supervised classifier (e.g. **XGBoost/LightGBM** classifier). Boosting models are proven in credit risk settings <sup>1</sup>.

- **3. Transaction Anomaly/Fraud Detection (Unsupervised):**

- *Inputs:* Raw transaction features – e.g. txn amount, time of day, merchant category, balance after transaction, frequency patterns.
- *Outputs:* An anomaly score or flag for each transaction. (No direct “target” label required during training.)
- *Approach:* Unsupervised outlier models like **Isolation Forest** (or One-Class SVM / autoencoder) to flag sudden abnormal transactions.

- **4. Cash-Flow Forecasting (Time-Series Regression):**

- *Inputs:* Historical balance series (daily or weekly), salary cycles, recurring expenses, EMI dates.
- *Outputs:* Predicted account balance over the next 7–30 days and warning flags if balance may hit zero (EMI failure risk).
- *Approach:* Time-series models (e.g. **ARIMA** for simplicity, or **LSTM neural nets** for complex patterns).

- **5. Wellness Recommendation (Rule/Tree-based):**

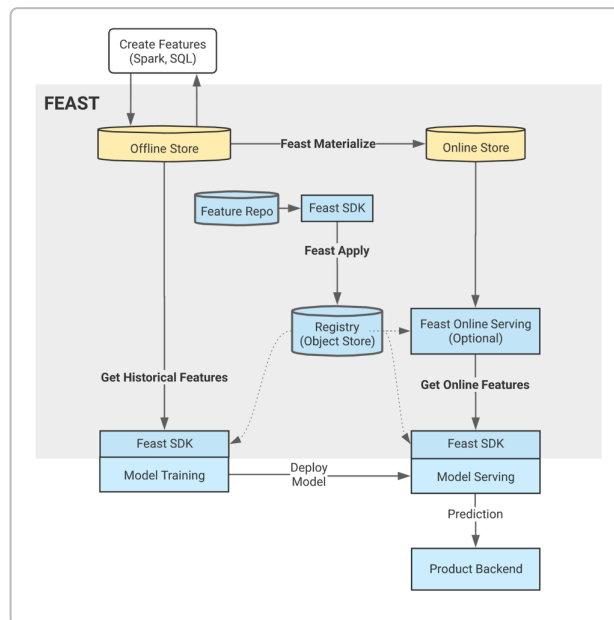
- **Inputs:** Summary financial metrics – e.g. savings rate, category spending ratios, debt ratio, current risk level.
- **Outputs:** Actionable advice (discrete categories) like “Reduce X spending” or “Increase savings by Y%”.
- **Approach:** A simple **decision tree or rule-based engine**, possibly with association-rule mining, to generate explainable recommendations.

These models are connected. For example, raw transactions feed the Health Score and Anomaly models; the Health Score feeds into Default Risk; risk outputs and spending trends feed the Wellness model. In this design, the **Health Score** and **Default Risk** models are core (must-have), while the **Forecast** and **Wellness** modules are advanced enhancements.

## Data Processing & Feature Pipelines

Raw financial data must be transformed into model-ready features. We propose the following data flow:

1. **Data Ingestion:** Collect raw inputs (bank statements, transactions, credit bureau data, etc.) into a data lake or database.
2. **Preprocessing:** Cleanse and standardize data (e.g. parse dates, handle missing values, encode categories).
3. **Feature Computation:** Aggregate transactions by period. For example, compute monthly totals for income, expenses, savings, EMI payments, and volatility measures. Derive ratios (EMI/Income, savings rate) and flags (e.g. “expense spike” boolean).
4. **Feature Store (Batch & Real-Time):** Store features in a structured form. A modern system uses a dual-store approach: an **offline store** (historical features for model training) and an **online store** (real-time features for live scoring). This ensures consistency between training and inference <sup>2</sup> <sup>3</sup>. In practice, batch jobs (e.g. nightly Spark jobs) populate the offline feature tables, while streaming processors update an in-memory online store with the latest values.



*Figure: A typical feature-store architecture (here using Feast): Spark/SQL jobs compute features into an offline store, and streaming/ingestion jobs update an online store. The offline store feeds model training, while the online store serves real-time inference <sup>2</sup> <sup>3</sup>. In FinPulse, transaction data is cleaned and aggregated daily (offline) and recent updates (e.g. latest balance) are kept online. This ensures training and live predictions use the same feature definitions.*

Such pipelines ensure repeatability and low-latency access. In summary, the end-to-end flow is: **Raw Data** → **ETL (cleaning/aggregation)** → **Feature Store** → **Model Input**.

## Model Training & Inference Deployment

For each model, we build separate training and inference pipelines:

- **Training Pipeline:** Uses historical data and the offline feature store. Steps include dataset assembly, train/validation split, model fitting (with hyperparameter tuning), and evaluation. For example, we might train the Health Score and Risk models on months of transactional data. We should track metrics like RMSE (for score) or AUC/KS (for default risk) over time. After training, models and their metadata go into a model registry. In practice, jobs can be orchestrated via Airflow/Prefect.
- **Inference Pipeline:** Runs in real time or batch to score new data. For online inference (e.g. computing current Health Score or flagging anomalies), the service fetches the latest features from the online store and applies the trained model. Each model is typically exposed as a microservice or API endpoint. For robust deployment, each model endpoint runs in its own container/runtime (so upgrades to one ML library don't break others) <sup>4</sup>. Monitoring modules track latency and accuracy drift.

**Model Choices:** In FinPulse's context, industry best-practices apply. For credit-related predictions, gradient boosting models (XGBoost/LightGBM) "perform excellently" on tabular risk data <sup>1</sup>, while logistic regression provides an interpretable benchmark. Unsupervised anomaly detection and simple decision trees require less "heavy" infrastructure.

Overall, this multi-pipeline architecture yields a scalable system. In fact, ML experts note that a well-designed architecture "helps teams build scalable, dependable, and efficient machine learning systems" <sup>5</sup> and can ensure the "integrity and safety of the ML infrastructure" <sup>6</sup>.

## API Endpoints & Contracts

Each model is exposed via a REST API endpoint. Input and output are JSON, often structured like Pandas data frames <sup>4</sup>. Example endpoints:

- **POST /predict/health\_score**  
Request JSON:

```
{
  "user_id": 12345,
  "features": {
    "monthly_income": 50000,
    "total_expense": 30000,
    "savings_rate": 0.20,
    "emi_income_ratio": 0.25,
    "cashflow_volatility": 0.15,
    ...
  }
}
```

#### Response JSON:

```
{
  "health_score": 78.3,
  "risk_label": "Medium"
}
```

#### • POST /predict/default\_risk

**Request JSON:** { "user\_id": 12345, "features": { "health\_score": 78.3, "missed\_emi": 0, ... } }

**Response JSON:** { "default\_probability": 0.12, "risk\_class": "Safe" }

#### • POST /detect/anomaly

**Request JSON:** { "user\_id": 12345, "transaction": { "amount": 12000, "time": "2026-03-12T14:55:00Z", ... } }

**Response JSON:** { "anomaly\_score": 0.87, "is\_anomalous": true }

#### • POST /forecast/balance

**Request JSON:** { "user\_id": 12345, "date": "2026-03-13", "balance\_history": [45000, 46000, 47000, ...] }

**Response JSON:** { "forecast\_7d": 12000.0, "forecast\_30d": 8000.0, "low\_balance\_risk": true }

#### • POST /recommend/wellness

**Request JSON:** { "user\_id": 12345, "features": { "savings\_rate": 0.12, "debt\_income\_ratio": 0.4, ... } }

**Response JSON:** { "advice": "Reduce discretionary spending by 10%", "recommendation\_code": "SAVE\_MORE" }

The API design treats each model as a function from input JSON→output JSON (akin to a Pandas DataFrame transform) <sup>4</sup>, which simplifies integration. All inputs and outputs should be versioned and documented.

# Combined End-to-End Workflow

In summary, FinPulse's architecture follows a layered ML pipeline (data→features→models→serve). The diagram below illustrates the full workflow. Raw transaction and profile data flows into data processing/feature pipelines, populating offline and online stores. Each ML service then ingests its features (from the stores) to generate predictions. Lenders and borrowers see results via separate dashboards.



Figure: Example end-to-end ML system architecture. Data is collected and preprocessed, features are computed (often stored in offline/online feature stores), models are trained and deployed, and predictions are monitored. This high-level workflow (adapted from common ML architecture patterns <sup>5</sup>) matches FinPulse's design: financial data → feature pipelines → model scoring → dashboards.

Overall, this modular architecture ensures robustness and scalability. As ML best-practice notes, a well-structured pipeline allows continual improvement and “makes the system efficient, maintainable, and scalable” <sup>6</sup>.

**Sources:** The design above combines FinPulse's requirements with standard ML pipeline patterns <sup>5</sup> <sup>7</sup>, feature store principles <sup>2</sup> <sup>3</sup>, and industrial model-serving guidance <sup>4</sup> <sup>6</sup>.

---

<sup>1</sup> <sup>2</sup> <sup>7</sup> How to Build a Risk-Scoring Engine: MLOps for Financial Services | APPIT Software Solutions  
<https://www.appitsoftware.com/blog/how-to-build-risk-scoring-engine-mlops-financial-services>

<sup>3</sup> Feature Stores: Architecture and Online/Offline Storage  
<https://www.dragonflydb.io/blog/feature-store-architecture-and-storage>

<sup>4</sup> Unifying Machine Learning Model Inference at Thumbtack | by Oleksandr Pryimak | Thumbtack Engineering | Medium  
<https://medium.com/thumbtack-engineering/unifying-machine-learning-model-inference-at-thumbtack-eacb08d11680>

<sup>5</sup> <sup>6</sup> Machine Learning Architecture Diagram: Key Components  
<https://lakefs.io/blog/machine-learning-architecture-diagram/>